

CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

From Java to C

Why C?

In General

- Efficient: small executables and low memory overhead
- Low-Level Control: fine-tune performance and memory
- Portable: Runs on every type of computer, from microcontrollers to supercomputers

In this Class

- Closer to hardware: Learn how memory, variables, and functions map to the machine

Demo: Hello, World!

Demo: Java vs. C

Java vs. C

C:

```
#include<stdio.h>
void main() {
    printf("Hello, World!\n");
}
```

Java:

```
public class Hello {
    public static void main(String[] args) {
        System.out.printf("Hello, World!");
    }
}
```

Java vs. C

C:

```
#include<stdio.h>
void main() {
    printf("Hello, World!\n");
}
```

Like Java, all C programs start with the first line of the function called `main`

Java:

```
public class Hello {
    public static void main(String[] args) {
        System.out.printf("Hello, World!");
    }
}
```

Java vs. C

C:

```
#include<stdio.h>
Void main() {
    printf("Hello, World!\n");
}
```

Unlike Java, C is not object oriented so there are no classes

Java:

```
public class Hello {
    public static void main(String[] args) {
        System.out.printf("Hello, World!");
    }
}
```

Java vs. C

C:

```
#include<stdio.h>
void main() {
    printf("Hello, World!\n");
}
```

In C, all functions must be defined in the file where it is called, so we include the library where printf is defined into our file

Java:

```
public class Hello {
    public static void main(String[] args) {
        System.out.printf("Hello, World!");
    }
}
```

C Basics

Compiling C

We will use the GNU C Compiler:

```
gcc <options> filename.c
```

Option to specify output file: -o outputFile

Compiling C

We will use the GNU C Compiler:

```
gcc <options> filename.c
```

Option to specify output file: -o outputFile

```
Ex: gcc -o hello hello.c
```

Running C

Run the compiled program directly in the terminal using its name:

```
./hello
```

Declaring Variables

Like Java, variables and their types must be declared before use:

```
int var;
```

Variables can also be declared and initialized together:

```
int var = 5;
```

Data Types

C	Java	Description
char	byte	single character ASCII code
short	short	short integer
int	int	integer
long	long	long integer
float	float	single precision float
double	double	double precision float

Integer types exist in both Java and C.

Data Types

[illegible]

Booleans exist in Java, but not C.

Data Types

C	Java	Description
---	------	-------------

Unsigned integers and references exist in C, but not Java.

unsigned char		very short positive integer
unsigned short		short positive integer
unsigned int		positive integer
unsigned long		long positive integer
*		reference

Data Types

C	Java	Description
char	byte	single character ASCII code
short	short	short integer
int	int	integer
long	long	long integer
float	float	single precision float
double	double	double precision float
	boolean	true/false
unsigned char		very short positive integer
unsigned short		short positive integer
unsigned int		positive integer
unsigned long		long positive integer
*		reference

Data Type Size

In C, the size of data types depends on the machine that the code is compiled on.

Use `sizeof(dataType)` to compute the size of a data type in bytes at compile time.

Operators

Category	Operators	Difference from Java
Arithmetic	+ - * / %	n/a
Assignment	= += -= *= /= %=	c implicitly casts a wider data type to a narrower data type
Increment/Decrement	++ --	n/a
Comparison and Logical	< <= > >= == != && !	return type is an int
Ternary (Conditional)	?	n/a
Comma	,	evaluated left to right
Size	sizeof(expression)	returns size of type or variable in bytes

Implicit Casting (Assignment)

On assignment, C will implicitly cast a wider data type to a narrower.

```
int x = 3.0;
```

Compilation error in Java

In C, the value is automatically changed to 3

Implicit Casting (Assignment)

On assignment, C will implicitly cast a wider data type to a narrower.

```
int x = 3.0;
```

Compilation error in Java

In C, the value is automatically changed to 3

Be Careful:

```
int x = 3.5; // the value is 3
```

Implicit Casting (Arithmetic)

For arithmetic, if the operands have different data types, C will implicitly cast the operand with the narrower data type to the same type as the wider operand.

(same as Java)

```
float x = 3.5 + 10; // value is 13.5
```

Output

Use `printf()` to print a string to the terminal.

*very similar to Java's `System.out.printf`

```
printf("format string", v1, v2, ...);
```

Output

Use `printf()` to print a string to the terminal.

**very similar to Java's `System.out.printf`*

```
printf("format string", v1, v2, ...);
```

```
#include <stdio.h>
int main(){
    int x = 5;
    printf("integer: %d\n", x);
}
```

Input

Use `scanf()` to read input from the terminal.

*not at all similar to Java's Scanner

```
scanf("format string", &v1, &v2, ...);
```

Input

Use scanf() to read input from the terminal.

*not at all similar to Java's Scanner

```
scanf("format string", &v1, &v2, ...);
```

```
#include <stdio.h>
int main(){
    int i;
    scanf("%d", &i);
}
```

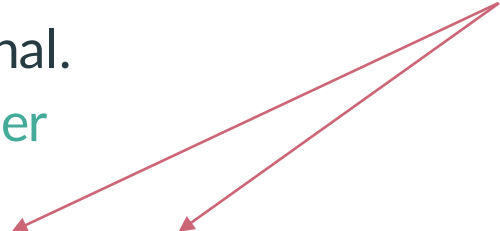
Input

Use scanf() to read input from the terminal.

*not at all similar to Java's Scanner

scanf("format string", &v1, &v2, ...);

We'll learn why later



```
#include <stdio.h>
int main(){
    int i;
    scanf("%d", &i);
}
```

Format Specifiers and Modifiers

Specifier	Data Type
%d	signed decimal integer
%u	unsigned integer
%f	floating point (float or double)
%c	char
%s	string

Modifier	Explanation
%4d	minimum 4 char width, right aligned
%-4d	minimum 4 char width, left aligned
%.2f	2 digits after decimal point
%4.2f	minimum 4 char width, 2 digits after decimal point

Exercise: calc.c